



Tajamul Khan

Optimize Python Code



@Tajamulkhann



Techniques

- Use List Comprehensions instead of Loops
- Use Built-in Functions
- Use Generators instead of Lists for large data
- Avoid Unnecessary Variables
- Use multiprocessing for Parallel Execution



Use List Comprehension

Loops can be slow, especially when handling large data. List comprehensions are faster and more concise. Example



 *Using a loop (slow)*

```
squares = []  
for i in range(10):  
    squares.append(i ** 2)
```



 *Using list comprehension*

```
squares = [i ** 2 for i in range(10)]
```



@Tajamulkhann



Use Built-In Functions

Python's built-in functions are optimized in C and run faster than manual implementations.

● ● ● ❌ *Manual summation (slow)*

```
total = 0
for num in range(1000):
    total += num
```

● ● ●

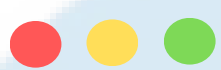
✅ *Using sum()*

```
total = sum(range(1000))
```



Use Generators

Generators save memory by yielding items one by one instead of storing them all in memory.



 *Using a list (high memory usage)*

```
nums = [i for i in range(10**6)] # Stores all
values in memory
print(sum(nums))
```



 *Using a generator (efficient memory usage)*

```
def generate_numbers(n):
    """Generator function that yields numbers from 0 to n-1"""
    for i in range(n):
        yield i # Yields values one by one instead of
storing them in memory
# Using the generator function
nums = generate_numbers(10**6) # Generator object
print(sum(nums))
# Computes sum efficiently without high memory usage
```



@Tajamulkhann



Reduce Variables

Reducing redundant variables minimizes memory usage.



Using an extra variable

```
text = "Python Optimization"  
length = len(text)  
print(length)
```



Directly using the function

```
print(len("Python Optimization"))
```



@Tajamulkhann



Use Multiprocessing

For CPU-intensive tasks, using multiple cores improves efficiency.



Multiprocessing

```
from multiprocessing import Pool
def square(n):
    return n * n
with Pool(4) as p: # Using 4 processes
    result = p.map(square, range(10))
print(result)
```

- **Memory Efficient :**

Generates values without storing them.

- **Lazy Evaluation :**

Computes only when needed.

- **Flexible :**

Easily extendable without extra memory usage.



@Tajamulkhann



Found Helpful?

Repost



Follow for more!

